Stock Forecasting with Time Series and Bayesian Linear Regression Model

Team member: Jialin Weng, Mengyang Wang

Class: DS4420

Instructor: Dr. Eric Gerber

Date: April 16, 2026

I. Introduction and Literature Review

Predicting stock prices is a critical issue in finance. It is also a very challenging task. Stock prices fluctuate over time and are influenced by many factors. Therefore, making completely accurate predictions is difficult. But stock price forecasting is still valuable. It helps people study market trends and see whether data methods can find useful patterns in real stock data.

In this project, we investigate whether two methods covered in class can be used to predict the stock prices of major technology companies. We analyzed eight stocks: NVDA, AAPL, MSFT, AVGO, MU, ORCL, AMD, and TSM. We selected these companies because they are major technology firms with long historical stock price records. The data comes from Yahoo Finance. We used daily closing price data from January 1, 2015, to December 31, 2025.

We used two models in this project. The first model is the SARIMAX time series model. The second model is the Bayesian linear regression model. Both models use the same stock data. Our goal is to examine how each model performs on the same dataset.

Previous research in stock price forecasting has used both traditional statistical methods and newer machine learning methods. Traditional methods, such as ARIMA and simple regression models, were often used in earlier studies. These methods can perform well when data patterns are relatively stable and simple. However, stock market data is often more complex. Because of this, many recent studies have also tested machine learning and deep learning methods.

Reshma et al. (2025) studied stock price prediction with machine learning methods. The authors used a number of algorithms, i.e., SVM, Random Forest, and LSTM networks, to analyze the stock price history. The results suggest that machine learning is able to learn the tendencies within the financial data and make specific forecasts. This study is useful for our project because it shows that stock prediction can benefit from data based methods.

Lestari, Deviana, and Kusuma (2026) compared deep learning models for stock price prediction. Their study focused on LSTM and Bi LSTM models. They used MAE and RMSE to measure model performance. These are also common measures in forecasting studies. Their results showed that LSTM gave more reliable predictions than Bi LSTM in their setting. This study is useful for our project because it shows that different models can give different forecasting results on stock data.

Poonkodi et al. (2026) compared ARIMA, SARIMA, and SARIMAX models for financial forecasting using NSE Nifty and BSE Sensex data. Their results showed that SARIMAX performed better than ARIMA and SARIMA in their setting, partly because it could include external variables. This study is useful for our project because it supports the use of SARIMAX in financial time series prediction.

Overall, past studies show that different quantitative methods can be useful for stock price prediction. Time series models are useful for modeling time dependence in stock prices. Bayesian linear regression models are useful for using lagged predictors and for showing uncertainty in the model. Based on these ideas, our project studies stock price prediction for eight large technology companies by using Yahoo Finance daily price data from 2015 to 2025. By applying both a SARIMAX time series model and a Bayesian linear regression model, we aim to examine how these two methods perform on the same stock prediction task.

II. ML Methodology and Data Collection

Time Series Model

For the time series part of this project, we used daily stock price data from Yahoo Finance. We selected eight well-known technology companies: NVDA, AAPL, MSFT, AVGO, MU, ORCL, AMD, and TSM. The data period was from January 1, 2015 to January 1, 2026, so the full year of 2025 was included. After downloading the data, we kept the daily closing prices, removed rows with missing values, and sorted the data in time order.

Before fitting the model, we changed the stock prices into log prices. This helped reduce the scale difference among different stocks and made the price series easier to model. After that, we made lagged predictors for our stocks. For each target stock, we calculated one-day lagged 20-day moving average and one-day lagged 50-day moving average. For the other seven stocks, we created one-day lagged log prices. Lagged variables let the model use earlier data instead of current data. The model took into account the stock's historical movement and the lagged effect of other technology stocks.

Each stock has their own SARIMAX models. The dependent variable was the current log price of the target stock. The explanatory variables were the lagged log prices of the other seven stocks and the lagged moving averages of the target stock. The model must account for temporal dependency (within the same stock) and cross-sectional dependence (of huge technological stocks), hence this technique is used. Because many of these firms are in the same industry, their stock prices may move in sync.

Splitting the data into training and testing datasets allowed the model to learn. A training dataset was produced using the first 80% of observations and a test dataset using the remaining 20%. The time series split keeps the time data, which makes the forecasting problem like one in real life. The Augmented Dickey-Fuller test was used for stationarity testing and diagnostics on the training set before model fitting. Next, we used auto_arima to find the non-seasonal ARIMA order for each stock. This allowed each stock to have its own ARIMA order.

After defining order and lagged exogenous regressors, we applied the model to training data to create a SARIMAX model. We then used the model to predict test data. Since the model used log prices, we changed the forecasts back to the original price scale by using the exponential function. RMSE and MAE are effective methods for evaluating stock price forecasts using test data.

This time series approach often allowed for systematic projections of stock values using historical data. SARIMAX employed a time-series structure, lagged peer-stock prices, and moving averages. This lets us see whether peer stocks and moving averages might better predict the stock prices of big tech companies.

Bayesian Linear Regression Model

In the Bayesian portion of this project, we used daily stock price data from Yahoo Finance. We selected eight major technology companies: NVDA, AAPL, MSFT, AVGO, MU, ORCL, AMD, and TSM. These companies were chosen because they are major players in the technology industry and have long price histories. The sample time range is from January 1, 2015, to January 1, 2026. After downloading the data, we retained the daily closing prices, merged the data for these eight stocks by date, and sorted them in chronological order.

To prepare the data, we first constructed the log prices for each stock. Then, for each target stock, we used only past information to construct the predictor variables. We used the one-day lagged logarithmic price of the target stock. We also used the one-day lagged 20-day moving average and 50-day moving average of the target stock. Additionally, we used the one-day lagged logarithmic prices of the other seven stocks. In this way, the model can utilize both the stock's own historical patterns and information from related stocks.

We built one Bayesian linear regression model for each stock. The target variable was the current log price of the chosen stock. The input variables were the lagged price of the target stock, lagged moving averages, and the one day lagged log prices of the other seven stocks. We also added an intercept term. We split the data by time. The first 80% was used for training. The last 20% was used for testing. This made the prediction task more real.

We used Gibbs sampling to fit the model. The sampler ran for 4,000 iterations. The first 1,000 iterations were removed as burn-in. The other samples were used to get the posterior mean of the regression coefficients and the posterior distribution of σ^2. For prediction, we generated posterior predictive draws on the log scale. These draws used the sampled regression coefficients and sampled σ^2 values. Then we used the average of the predictive draws as the forecast. After that, we changed the forecasts back to the original price scale by using the exponential function. We used RMSE and MAE on the original price scale to check model performance. Overall, this model predicts stock prices in a clear Bayesian way by using past prices, moving averages, and information from other stocks.

III. ML Results and Interpretation

Time Series Model

Final Summary Table:

	Stock	ADF Statistic	ADF p-value	Order	RMSE	MAE
0	NVDA	-1.166688	0.687759	(2, 0, 2)	8.454404	6.666853
1	AAPL	-0.333347	0.920647	(1, 0, 0)	8.703246	6.840120
2	MSFT	-1.009397	0.749822	(2, 0, 1)	14.765108	11.339033
3	AVGO	-0.435281	0.904039	(1, 0, 2)	18.241702	12.986977
4	MU	-1.070114	0.726842	(3, 0, 2)	10.542234	8.182952
5	ORCL	-0.398457	0.910356	(2, 0, 2)	14.840024	10.242821
6	AMD	-1.336050	0.612522	(1, 0, 2)	15.937639	12.662384
7	TSM	-1.067145	0.727995	(1, 0, 0)	12.934177	9.981193

Table 1. SARIMAX Results for Eight Technology Stocks

Table 1 presents the main results of the SARIMAX model for all eight technology stocks. The selected ARIMA order was chosen separately for each stock by auto_arima, so the model structure was allowed to differ across companies. Forecast performance also varied across stocks. The RMSE and MAE for NVDA were 8.45 and 6.67, respectively, which were the lowest. The RMSE and MAE for AAPL were 8.70 and 6.84, respectively, which were also pretty good. On the other hand, AVGO and AMD had bigger forecast errors, which means that these stocks were harder to predict in this model. The table also reports the ADF p-values for the training series. These values were used as a diagnostic check for stationarity before model fitting. For this reason, the ADF test was used mainly as a diagnostic check, while the final ARIMA order was selected automatically by auto_arima.

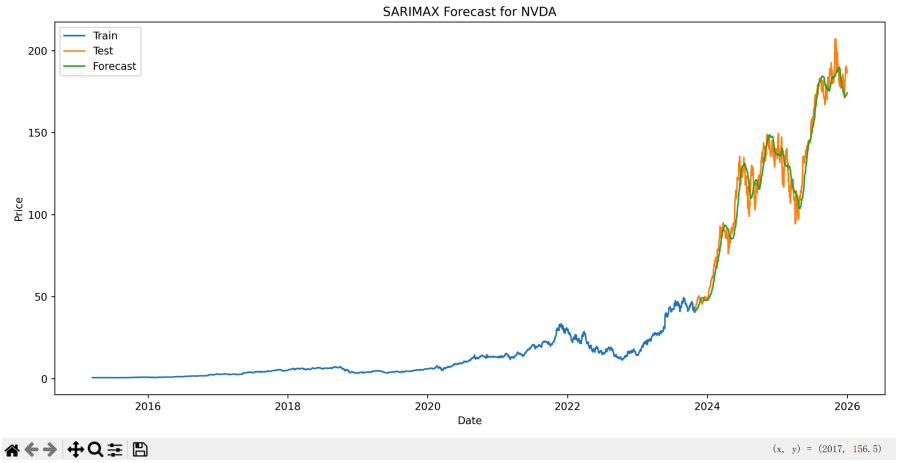

Figure 1. SARIMAX Forecast for NVDA

Figure 1 shows the forecast result for NVDA, which was one of the best-performing stocks in the model. The forecast line follows the main pattern of the testing data reasonably well and captures the overall upward and downward movement during the test period. The predicted series is smoother than the actual test series, so it does not match every short-term fluctuation exactly. However, it still tracks the general trend clearly, which is consistent with the relatively low RMSE and MAE values reported for NVDA in Table 1.

Overall, the SARIMAX model produced useful forecasts, but the model did not perform equally well for all stocks. It worked better for NVDA and AAPL, while AVGO and AMD showed larger forecast errors. This suggests that some stocks had patterns that were easier for the model to capture, while others were more difficult to predict using lagged peer prices and lagged moving averages alone.

Bayesian Model

Bayesian Linear Regression Summary:

	Stock	Posterior mean of Sigma²	RMSE	MAE
1	AAPL	0.001264	3.642979	2.534997
2	NVDA	0.001850	3.810938	2.770185
3	MU	0.001777	4.649755	3.132718
4	TSM	0.001304	5.251451	3.812865
5	AMD	0.002293	5.459525	3.679547
6	MSFT	0.001231	6.027488	4.419238
7	ORCL	0.001206	6.200605	3.298086
8	AVGO	0.001408	7.455347	4.863672

Table 2. Bayesian Linear Regression Results for Eight Technology Stocks

Table 2 shows the main results of the Bayesian linear regression model for all eight technology stocks. The forecast results were different for each stock. AAPL had the lowest forecast error. RMSE was 3.64, and MAE was 2.53. NVDA also had good results. RMSE was 3.81, and MAE was 2.77. MU and TSM had medium forecast accuracy. AVGO had the largest forecast error. The RMSE was 7.46, and the MAE was 4.86. Overall, the results show that the Bayesian linear regression model made good predictions for all eight stocks. However, the predictions were not always correct for each company. The table also shows the posterior mean of Sigma² for each stock. This value shows the estimated error size in the Bayesian model.

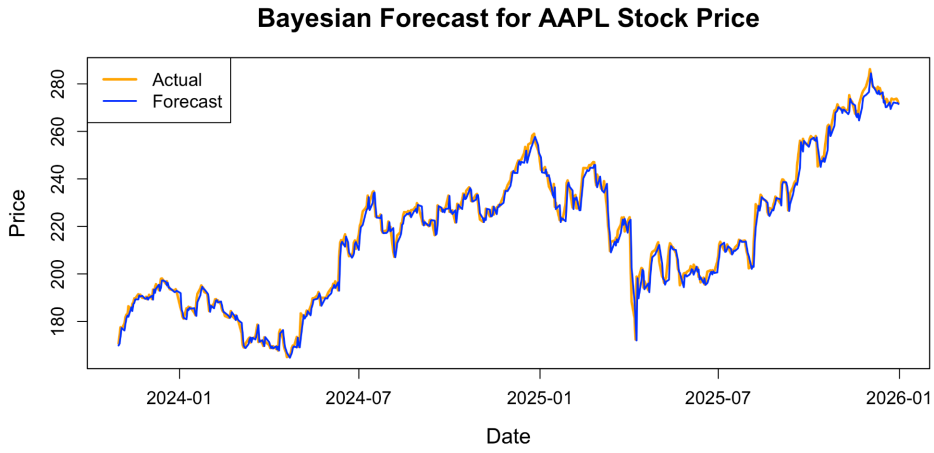

Figure 2. Bayesian Forecast for AAPL

Figure 2 shows the forecast result for AAPL, which was the best performing stock in the Bayesian model. The forecast line stays very close to the actual test series and follows the main upward and downward movement during the test period. The predicted series also matches many short term changes well, so the gap between the forecast and the actual price is small in most parts of the figure. This is consistent with the relatively low RMSE and MAE values reported for AAPL in Table 2.

Overall, the Bayesian model produced useful forecasts, but the forecasting performance was not the same across all stocks. It worked especially well for AAPL and NVDA, while AVGO showed larger forecast errors. This suggests that the Bayesian model was able to capture the price pattern of some stocks more effectively than others when lagged own stock prices, moving averages, and lagged peer stock prices were used as predictors.

IV. Conclusions and Future Work

In this project, two models were used to predict stock prices for eight large technology companies: a SARIMAX time series model and a Bayesian linear regression model. For the SARIMAX model, forecast accuracy differed across the eight stocks, which suggests that the time series model was able to capture useful short-run patterns for some stocks more successfully than for others. In this model, stock prices were first transformed to the log scale, the SARIMAX model was fitted using lagged peer-stock information and lagged moving averages as exogenous variables, and the forecasts were then converted back to the original price scale for evaluation.

The Bayesian linear regression model also produced useful forecasts for all eight stocks, and its forecasting performance also varied across companies. In this model, the regression was also fitted on log-transformed stock prices. The predictors included the lagged value of the target stock, lagged moving averages of the target stock, and lagged peer-stock prices. Posterior predictive values were generated on the log scale and then converted back to the original price scale for evaluation.

Both models were helpful for our project, although they weren't ideal for the model assumptions. The SARIMAX model is based on the idea that the ARIMA structure and the lagged explanatory variables can represent the temporal pattern. We used the ADF test as a diagnostic check for stationarity, but stock prices are not always fully stationary. The Bayesian linear regression model assumes that there is a linear connection between the lagged predictors and the current log price. This means that it may not be able to capture all of the nonlinear changes in the market. Also, both models solely utilized information about prices from the past. They didn't include news on profits, macroeconomic issues, market mood, or changes in interest rates. Because of this, the study was useful for examining the performance of two statistical models on the same stock data, but the findings should not be seen as flawless forecasts of what would happen in the market. In the future, we may add additional market variables, utilize stock returns, or test more flexible nonlinear approaches.

Overall, the SARIMAX model gave a useful time series forecasting framework. The Bayesian linear regression model gave another view based on regression under the same data setting.

V. Appendix

GitHub repo link:https://github.com/Jialin-1290/DS4420-Project

Appendix A. Python Training Implementation for SARIMAX

The following code shows the training part of the SARIMAX model.

```
import numpy as np
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
from pmdarima import auto_arima


# Define a function to run one SARIMAX model for one stock.
def run_sarimax_for_stock(stock_name, full_data, ticker_list,
train_ratio=0.8):
    """
    Run one SARIMAX model for one selected stock.


    Parameters:
    stock_name : str
        The stock ticker that we want to predict.
    full_data : pandas.DataFrame
        The full stock price data for all selected stocks.
    ticker_list : list
        The list of all stock tickers used in the model.
    train_ratio : float, optional
        The ratio of data used for training.
        The default value is 0.8.
    """

    # Take log of the stock prices.
    # This can help make the series more stable.
    log_data = np.log(full_data.copy())

    # Create lagged moving average variables for the target stock.
    # shift(1) means only past information is used.
```

```
    log_data[f"{stock_name}_MA20_lag1"] =
log_data[stock_name].rolling(20).mean().shift(1)
    log_data[f"{stock_name}_MA50_lag1"] =
log_data[stock_name].rolling(50).mean().shift(1)

    # Create one-day lagged variables for the other stocks.
    # These are used as explanatory variables.
    peer_stocks = [stock for stock in ticker_list if stock != stock_name]

    for stock in peer_stocks:
        log_data[f"{stock}_lag1"] = log_data[stock].shift(1)

    # Drop missing rows after creating lagged variables.
    log_data = log_data.dropna()

    # Set the target variable.
    # This is the log price of the selected stock.
    target = log_data[stock_name]

    # Set the explanatory variables.
    # These include lagged peer stock prices
    # and lagged moving averages of the target stock.
    exog_columns = [f"{stock}_lag1" for stock in peer_stocks]
    exog_columns += [f"{stock_name}_MA20_lag1", f"{stock_name}_MA50_lag1"]
    exog = log_data[exog_columns]

    # Split the data and keep the training part for model fitting.
    # 80% is used for training.
    train_size = int(len(log_data) * train_ratio)
    y_train = target.iloc[:train_size]
    X_train = exog.iloc[:train_size]

    # Run the ADF test on the training series.
    # This is used to check stationarity.
    adf_result = adfuller(y_train)
    adf_stat = adf_result[0]
    adf_pvalue = adf_result[1]
```

```
print("\n" + "=" * 70)
print(f"Running SARIMAX for {stock_name}")
print(f"ADF Statistic: {adf_stat}")
print(f"ADF p-value: {adf_pvalue}")

# Use auto_arima to choose the ARIMA order automatically.
# A non-seasonal model is used here.
auto_model = auto_arima(
    y_train,
    X=X_train,
    seasonal=False,
    trace=False,
    stepwise=True,
    error_action="ignore",
    suppress_warnings=True,
    max_p=3,
    max_q=3,
    max_d=2
)

order = auto_model.order
print(f"Selected order for {stock_name}: {order}")

# Fit the SARIMAX model using the training data.
model = SARIMAX(
    y_train,
    exog=X_train,
    order=order,
    enforce_stationarity=False,
    enforce_invertibility=False
)

results = model.fit(disp=False)
```

Appendix B. R Training Implementation for Bayesian Linear Regression

The following function shows the manual implementation of Bayesian linear regression with Gibbs sampling.

```
# Fit Bayesian linear regression with Gibbs sampling
bayes_linear_regression <- function(X, y, tau2 = 100, alpha0 = 2, beta0 = 1,
                                    n_iter = 4000, burn_in = 1000) {
  # Number of rows and columns
  n <- nrow(X)
  p <- ncol(X)

  # Prior mean and prior variance
  b0 <- matrix(0, nrow = p, ncol = 1)
  V0 <- diag(tau2, p)
  V0_inv <- solve(V0)

  # Store draws
  beta_draws <- matrix(NA, nrow = n_iter, ncol = p)
  sigma2_draws <- numeric(n_iter)

  # Set starting values
  beta_curr <- matrix(0, nrow = p, ncol = 1)
  sigma2_curr <- 1

  # Compute X'X and X'y
  XtX <- t(X) %*% X
  Xty <- t(X) %*% y

  # Run Gibbs sampling
  for (iter in 1:n_iter) {
    # Get posterior variance and mean for beta
```

```
  Vn <- solve(V0_inv + XtX / sigma2_curr)
  bn <- Vn %*% (V0_inv %*% b0 + Xty / sigma2_curr)

  # Draw beta from the conditional posterior
  beta_curr <- MASS::mvrnorm(1, mu = as.numeric(bn), Sigma = Vn)
  beta_curr <- matrix(beta_curr, ncol = 1)

  # Get posterior shape and rate for sigma2
  alpha_n <- alpha0 + n / 2
  beta_n <- beta0 + as.numeric(t(y - X %*% beta_curr) %*% (y - X %*% beta_curr)) / 2

  # Draw sigma2 from the conditional posterior
  sigma2_curr <- 1 / rgamma(1, shape = alpha_n, rate = beta_n)

  # Save the draws
  beta_draws[iter, ] <- as.numeric(beta_curr)
  sigma2_draws[iter] <- sigma2_curr
 }

 # Remove burn-in draws
 keep <- (burn_in + 1):n_iter

 # Return the main Bayesian results
 list(
  beta_draws = beta_draws[keep, , drop = FALSE],
  sigma2_draws = sigma2_draws[keep],
  posterior_mean = matrix(colMeans(beta_draws[keep, , drop = FALSE]), ncol = 1)
 )
}
```

References

Lestari, I. G. A. N., D. Deviana, and T. M. Kusuma. 2026. “A Comparative Analysis of Deep Learning Models for Stock Price Prediction.” *INOVTEK Polbeng – Seri Informatika*. https://jurnal.polbeng.ac.id/index.php/ISI/article/view/1496/657.

Poonkodi, M., M. Premalatha, A. A. Dixit, K. Yadav, and P. Sharma. 2026. “Comparative Analysis of NSE Nifty and BSE Sensex Forecasting Using ARIMA, SARIMA and SARIMAX Models.” In *Intelligent and Sustainable Systems: AI, Green IoT, and Adaptive Automation in Electrical and Communication Technologies*, edited by A. Punitha, S. Syedakbar, and S. Jeyasudha, 164–170. Boca Raton, FL: CRC Press. https://doi.org/10.1201/9781003773801-28.

Reshma, S., G. Tulasikrishna, C. S. Prashanth, C. Rakesh, K. Venkatesh, and K. S. R. Kumar. 2025. “Real-Time Stock Price Prediction and Market Analysis Using Machine Learning.” In *Proceedings of the International Conference on Research and Development in Information, Communication, and Computing Technologies*. https://www.scitepress.org/Papers/2025/139421/139421.pdf.